

ClubHouse

Deploy Guide: Vercel (frontend) + Render (API) + Supabase (Postgres)

REPO: github.com/harsha-2008/-ClubHouse (branch **main**) | 20 September 2026

This guide matches the config already in your repo: **vercel.json** (Vite build + SPA rewrite), **render.yaml** (API blueprint), **main.tsx** reading **VITE_API_URL**, and env-based admin bootstrap (**ADMIN_EMAILS**). Follow the parts in order: Supabase first, then Render, then Vercel, then Clerk, then verify.

Piece	Service	What it runs
Database	Supabase (free)	Postgres 16, all 18 tables via drizzle push
API	Render web service (free)	Express bundle dist/index.mjs, /api/*, health /api/healthz
Frontend	Vercel (free)	Vite static build artifacts/clubhouse/dist, calls API via VITE_API_URL
Auth	Clerk (existing keys)	Same publishable key on web; publishable + secret on API

Contents

- Part 0 — Prerequisites & deploy order
- Part 1 — Supabase database + schema push
- Part 2 — Render API service
- Part 3 — Vercel frontend
- Part 4 — Clerk domains
- Part 5 — Admin bootstrap (first login)
- Part 6 — End-to-end verification
- Troubleshooting & free-tier limits
- Appendix — exact env tables & commands

Part 0 — Prerequisites & deploy order

- GitHub repo pushed (done: **main** is on GitHub, includes **vercel.json** + **render.yaml**).
- Accounts: Supabase, Render, Vercel (all free), plus your existing Clerk account/keys.
- Your Clerk **publishable key** (pk_...) and **secret key** (sk_...) at hand.
- Local machine has Node 24 + pnpm (for the one-time schema push).
- Deploy in this order — each step needs the previous one: **Supabase** → **Render** → **Vercel** → **Clerk** → **verify**.

Why this order: Render needs the Supabase DATABASE_URL; Vercel needs the Render API URL (VITE_API_URL is baked in at build time, so set it **before** deploying the frontend).

Part 1 — Supabase database + schema push

1. Supabase Dashboard → New project. Name it **clubhouse**, save the database password, pick the region closest to you (ideally the same region you will pick on Render). Wait for provisioning.

2. Project Settings → Database → copy the **Session pooler** connection string (port 5432, looks like `postgresql://postgres.<ref>:<password>@aws-0-<region>.pooler.supabase.com:5432/postgres`). Use it verbatim — it is IPv4-compatible and supports prepared statements, which the Drizzle client requires. (Do NOT use the Transaction pooler on port 6543.)

3. Push the schema from your Windows machine (PowerShell, in the repo root):

```
$env:DATABASE_URL='postgresql://postgres.<ref>:<password>@aws-0-<region>.pooler.supabase.com:5432/postgres'
pnpm --filter @workspace/db run push
Remove-Variable DATABASE_URL
```

4. Verify in Supabase → Table Editor: you should see **users, colleges, college_members, college_join_requests, moderator_applications, college_meetings, clubs, club_members, club_management_events, projects, project_members, project_applications, project_messages, project_events, club_events, event_registrations, notifications, admin_audit_logs** (18 tables).

Part 2 — Render API service

Your repo already contains **render.yaml** (Node 24, build/start commands, **/api/healthz** check), so use the Blueprint flow:

- Render Dashboard → New → **Blueprint** → select the **-ClubHouse** repo. Render detects **render.yaml** and creates the **clubhouse-api** web service.
- Plan: **Free**. Region: same as Supabase if possible.
- Fill the prompted env vars (see Appendix table): **DATABASE_URL** (Session pooler string from Part 1), **CLERK_PUBLISHABLE_KEY**, **CLERK_SECRET_KEY**, **ADMIN_EMAILS** (your own login email — temporary, removed in Part 5). **PORT** is supplied by Render automatically; the server reads it.
- Deploy and wait. Open **`https://<your-api>.onrender.com/api/healthz`** → expect `{"status": "ok"}`. Also check **/api/readyz** (proves the DB connection works). Copy the API origin (e.g. `https://clubhouse-api-xxxx.onrender.com`) — no trailing slash — you need it for Vercel.

Free-tier note: Render free sleeps after ~15 min idle, so the first request after idle takes ~30–60 s (cold start). This is normal — the frontend will just feel slow once, then fast.

Part 3 — Vercel frontend

Your repo already contains **vercel.json** (builds **artifacts/clubhouse**, outputs **dist**, SPA fallback to `index.html`) and **main.tsx** already points API calls at **VITE_API_URL**. So:

- Vercel Dashboard → Add New Project → import **-ClubHouse**. Keep Root Directory as the repo root — **vercel.json** already sets the build (`pnpm --filter @workspace/clubhouse run build`) and output (`artifacts/clubhouse/dist`).
- Environment Variables (Production): **VITE_API_URL**=your Render origin (no trailing slash), **VITE_CLERK_PUBLISHABLE_KEY**=your pk_... key. (Vite bakes `VITE_*` in at build time — set them **before** deploying; changing them later needs a redeploy.)
- Deploy. Open the Vercel URL — the landing page loads; sign-in calls go to Render via **VITE_API_URL**.

Part 4 — Clerk domains

- Clerk Dashboard → add your Vercel domain (e.g. `clubhouse-xxx.vercel.app`) to allowed origins / redirect allowlist, same for your Render domain if you ever open API pages directly.
- Keep using the same publishable key everywhere; the secret key lives **only** on Render, never in the frontend.

Part 5 — Admin bootstrap (first login)

- With **ADMIN_EMAILS** set on Render to your login email, sign up on the Vercel site and complete the 2-step onboarding. Your profile is created with **role=admin** (one time only).

- Confirm: sign in → open **/admin** — the dashboard loads.
- **Immediately remove ADMIN_EMAILS** from Render env vars and redeploy. From then on admin rights come only from the database role — nobody else can self-promote.

Part 6 — End-to-end verification

- **/api/healthz** and **/api/readyz** return status ok.
- Sign up → onboarding (2 steps) → dashboard loads with stats.
- Discover colleges → join request; create project → apply/invite → project chat sends; schedule a meeting/event; register for an event.
- Notifications bell shows entries; profile edit saves; **/admin** approves a college.
- If all pass, production is live. Congratulations.

Troubleshooting & free-tier limits

Symptom	Cause / fix
First API call slow (~60 s)	Render free cold start — normal. Subsequent calls are fast.
Frontend shows network/API errors	VITE_API_URL missing/wrong or has trailing slash; fix in Vercel env and redploy (build-time var).
Blank page on refresh of /dashboard etc.	SPA fallback missing — your vercel.json rewrite <code>/(.*) → /index.html</code> covers this; redeploy if edited.
DB errors / 500 on API	Wrong DATABASE_URL (use Session pooler :5432 verbatim) or schema not pushed — rerun Part 1 step 3.
Supabase says project paused	Free Supabase pauses after ~1 week idle — Dashboard → Restore/Resume, no data loss.
Clerk sign-in loops or origin error	Vercel domain not in Clerk allowlist — add it (Part 4).
File uploads fail (club logo/brochure)	Expected: uploads use the Replit GCS sidecar, which does not exist on Render. Avatars via Clerk still work. Ask for the GCS service-account migration to enable uploads.
429 Too many requests	Rate limiter (20 writes/min) — slow down; raise limits in <code>src/lib/rateLimit.ts</code> if needed.

Appendix — exact env tables & commands

Render (API service) — Production

Variable	Value / source
DATABASE_URL	Supabase Session pooler URI (Part 1). REQUIRED.
CLERK_PUBLISHABLE_KEY	Clerk Dashboard → API keys (pk_...). REQUIRED.
CLERK_SECRET_KEY	Clerk Dashboard → API keys (sk_...). REQUIRED (proxy + server auth).
ADMIN_EMAILS	Your login email. TEMPORARY — remove after Part 5.
NODE_ENV / LOG_LEVEL / NODE_VERSION	production / info / 24 (in render.yaml already).
PORT	Provided by Render automatically — do not set.

Vercel (frontend) — Production

Variable	Value / source
VITE_API_URL	Render API origin, e.g. <code>https://clubhouse-api-xxxx.onrender.com</code> (NO trailing slash). REQUIRED.
VITE_CLERK_PUBLISHABLE_KEY	Same pk_... as Render. REQUIRED.

Handy commands (repo root)

```
pnpm --filter @workspace/db run push    # with DATABASE_URL set – create all tables
pnpm run typecheck    # must be green before redeploys
pnpm --filter @workspace/api-spec run codegen    # after any openapi.yaml edit
curl https://<api>.onrender.com/api/healthz    # expect {"status":"ok"}
```

— End of guide — Repo: github.com/harsha-2008/ClubHouse (main) —